

设计模式篇期末复习

依据：设计模式、详细设计、模块化与信息隐藏课件，123.txt，语音版重点，考试题目类型与重点，2020/2022 往年卷。

使用建议

今年设计模式题采用诱导式问法，重点不是背出模式名称，而是根据材料识别变化点，设计稳定接口，用组合和多态隔离变化，并说明使用了哪些设计原则。

一、考试定位

优先级	内容	依据	典型产物
高	策略模式与支付/折扣/优惠券变化	2022 直接考微信、支付宝和未来支付方式	接口、具体策略、上下文类图 + 原则
高	模式背后的原则	答疑明确强调诱导式设计	OCP、DIP、SRP、面向接口、组合优先、封装变化
中高	抽象工厂/工厂方法与 DAO	2020 直接考数据库变化	工厂接口、DAO 接口、具体数据库实现
中	迭代器模式	课件重点；与信息隐藏结合	遍历集合但不暴露内部表示
中	单件模式	课件重点	唯一实例、私有构造器、全局访问点及风险

答题立场

不要看到 if/switch 就机械套模式。先判断它是不是稳定变化点、是否存在多个可替换实现、调用者是否应该不知道具体类型，再决定是否需要模式。

二、设计模式与可修改性

1. 什么是设计模式

设计模式是在特定上下文中，对反复出现的设计问题及其成熟解决方案的命名。它不是可直接复制的代码模板，而是关于职责、协作、依赖方向和变化封装的设计结构。

组成	要回答的问题
典型问题	什么变化导致当前设计难以维护？
设计分析	哪些部分稳定，哪些部分变化？
解决方案	由哪些抽象角色和具体角色协作？
后果	获得什么灵活性，又付出什么复杂度？

模式

- 典型问题
- 设计分析
- 解决方案
- 案例

课件截图：学习模式要从典型问题、设计分析、解决方案和案例展开。

2. 三种可修改性

类型	含义	例子
M 修改性	已有实现内部可以修改而不影响客户	修改 VIP 折扣算法
E 扩展性	可以增加新实现而不修改稳定客户	新增学生折扣
C 灵活性	运行时可以配置或替换实现	为订单动态选择优惠策略

可修改性

- (M)实现的可修改性
 - 对已有实现的修改
 - 例如：修改现有促销策略
- (E)实现的可扩展性
 - 对新的实现的扩展
 - 例如：增加一条新的促销策略
- (C)实现的灵活性
 - 对实现的动态配置
 - 例如：动态修改更改某商品对应促销策略

课件截图：可修改性分为修改、扩展和动态配置三个方向。

三、模式推导方法

- 1 从题目中圈出可能变化的名词和动词：支付方式、折扣算法、数据库、集合结构、对象创建。
- 2 找出稳定使用方式：所有支付都要 pay，所有折扣都要 apply，所有 DAO 都要 save/find。
- 3 把稳定行为提取为接口，把每种变化放入独立实现类。
- 4 让业务上下文持有接口引用，通过组合委托，不在内部判断具体类型。
- 5 把具体对象的选择或创建移到客户端、工厂或配置层。
- 6 验证新增类型时是否只需增加类；再说明 OCP、DIP、SRP 和组合优先。

模式题通用角色

角色	职责
抽象接口	定义稳定契约，隐藏具体实现差异
具体实现	封装一种算法、产品、访问或遍历方式
上下文/客户	依赖抽象，把请求委托给具体实现
创建者/配置者	决定选择哪个具体实现并完成注入

关键区别

业务上下文负责“使用策略”，客户端或工厂负责“选择策略”。如果上下文仍然通过 type + switch 创建具体类，变化只是换了位置，OCP 仍可能没有真正实现。

四、策略模式 Strategy

1. 解决什么问题

当同一行为有多个可替换实现，或一个类内部出现依据类型选择算法的 if/switch 时，可以把算法族分别封装，并让上下文通过统一接口使用。

```
interface PaymentStrategy {
    PaymentResult pay(double amount);
}

class WeChatPayment implements PaymentStrategy {
    public PaymentResult pay(double amount) { ... }
}

class RechargeService {
    private final PaymentStrategy payment;
    RechargeService(PaymentStrategy payment) { this.payment = payment; }
    PaymentResult recharge(double amount) { return payment.pay(amount); }
}
```

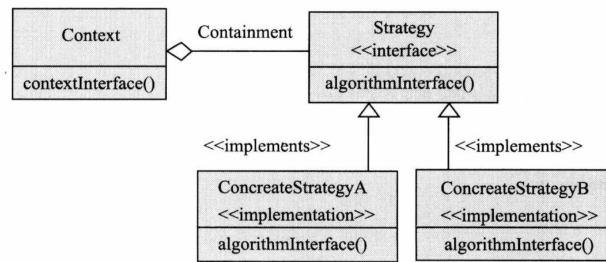


图 16-8 策略模式

策略模式：定义了算法族，分别封装起来，让他们之间可以互相替换，此模式让算法的变化独立于使用算法的客户。

策略模式类图

课件截图：策略模式定义算法族并分别封装，使算法变化独立于使用算法的客户。

2. 参与者与协作

角色	支付例子	职责
Strategy	PaymentStrategy	声明 pay 的稳定接口
ConcreteStrategy	WeChatPayment、AlipayPayment	实现具体支付算法/外部调用
Context	RechargeService	持有策略引用，把支付请求委托给策略
Client	Controller/配置/工厂	选择具体策略并注入上下文

3. 为什么符合设计原则

原则	体现
OCP	新增银行卡支付只增加 BankCardPayment，不修改 RechargeService
DIP	充值服务依赖 PaymentStrategy 抽象，而不是微信/支付宝具体类
SRP	每个策略只负责一种支付实现，上下文只负责充值流程
组合优于继承	上下文持有策略对象，运行时可替换
封装变化	把最可能变化的支付方式隔离在策略族中

4. 优点、代价与适用条件

方面	内容
优点	消除大段条件分支；算法可独立修改、测试和扩展；支持运行时切换
代价	类和对象数量增加；客户端必须知道如何选择策略；上下文与策略传递数据有成本
适用	多个相关类只在行为实现上不同；同一行为有多个变体；类型分支频繁变化
不适用	只有一个简单且稳定的实现，或差异不是可替换算法

五、工厂模式与抽象工厂

1. 为什么需要工厂

对象创建本身可能包含类型判断、配置、连接参数和产品组合。若客户到处直接 new 具体类，就会同时依赖创建细节和使用细节。工厂把创建变化集中封装，使客户只面对抽象产品接口。

```
interface UserDao {
    User findById(String id);
    void save(User user);
}

interface DaoFactory {
    UserDao createUserDao();
    RechargeRecordDao createRechargeRecordDao();
}

class MySqlDaoFactory implements DaoFactory { ... }
class SqlServerDaoFactory implements DaoFactory { ... }
```

2. 抽象工厂参与者

角色	DAO 例子	职责
AbstractFactory	DaoFactory	声明创建一组 DAO 产品的方法
ConcreteFactory	MySqlDaoFactory	创建同一数据库产品族
AbstractProduct	UserDao、RecordDao	声明数据访问接口
ConcreteProduct	MySqlUserDao 等	实现具体数据库访问
Client	Service	使用工厂和 DAO 接口，不知道具体数据库

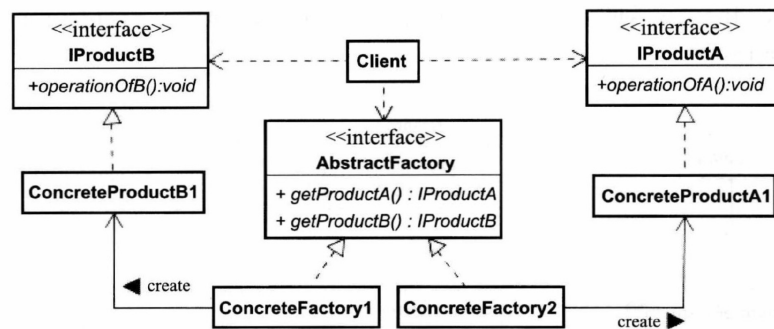


图 16-15 抽象工厂模式

抽象工厂模式：定义了一个创建对象的接口，由子类决定要实例化哪一个类。工厂方法让类的实例化延迟到子类。

抽象工厂模式的类图

课件截图：抽象工厂同时实现工厂多态与产品多态，隔离产品族创建。

3. 工厂方法与抽象工厂的区别

比较	工厂方法	抽象工厂
目标	延迟某一种产品的创建	创建一组相互配套的产品族
机制	通常通过继承，让子类决定实例化类型	通过工厂对象组合，提供多个创建方法
例子	createPayment()	同时 createUserDao()、createOrderDao()
扩展	新增具体产品实现较自然	新增产品族容易，新增产品种类需改工厂接口

4. 体现的设计原则

- OCP：新增数据库产品族可以新增具体工厂和 DAO 实现。
- DIP：业务层依赖 DAO/Factory 接口，而不是 MySQL 类。
- SRP：对象创建职责从业务逻辑中分离。
- 信息隐藏：数据库连接和实例化组合被封装在具体工厂内部。

常见误区

DAO 本身是数据访问抽象，不自动等于某个 GoF 模式。若题目强调数据库产品族整体切换，适合抽象工厂；只有单个 DAO 的创建变化，也可以使用工厂方法或简单工厂配合依赖注入。

六、单件模式 Singleton

单件模式确保一个类只有一个实例，并提供全局访问点。典型实现需要私有构造器、类内部保存唯一实例，以及公开获取实例的方法。

```
class ConfigRegistry {  
    private static final ConfigRegistry INSTANCE = new ConfigRegistry();  
    private ConfigRegistry() {}  
    public static ConfigRegistry getInstance() { return INSTANCE; }  
}
```

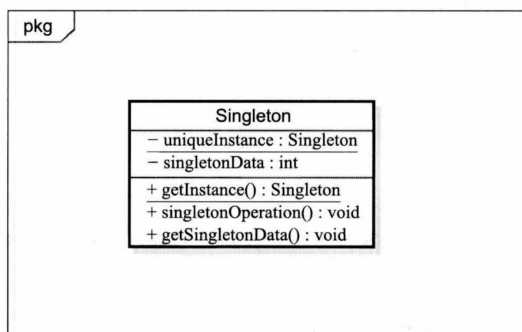


图 16-19 单件模式

单件模式确保一个类只有一个实例，并提供一个全局访问点。

单件模式类图

课件截图：单件模式确保一个类只有一个实例，并提供全局访问点。

优点	风险
控制实例数量；统一访问共享资源；可延迟初始化	本质接近全局状态；隐藏依赖；不易 Mock；并发和生命周期管理复杂

考试判断

只有业务上确实要求唯一实例时才使用。不要为了“方便访问”把普通服务做成 Singleton；依赖注入容器管理单例生命周期通常更清晰。

七、迭代器模式 Iterator

迭代器提供顺序访问聚合对象元素的方法，而不暴露其内部表示。调用者只知道 hasNext/next，不需要知道内部是 List、Set、树还是数据库游标。

```
Iterator<RechargeRecord> it = records.iterator();
while (it.hasNext()) {
    RechargeRecord record = it.next();
    process(record);
}
```

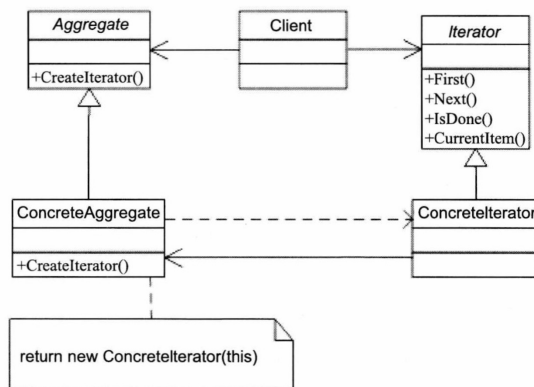


图 16-24 迭代器模式

迭代器模式提供一种顺序访问一个聚合对象的各个元素，而不暴露其内部表示。

迭代器模式类图

课件截图：迭代器隐藏聚合结构，使遍历算法与容器实现解耦。

解决的问题	体现的原则
遍历集合但不暴露 List/Set/数组等内部结构	信息隐藏、面向接口、DIP
希望替换聚合结构而不修改遍历客户	OCP、封装变化
限制客户直接修改内部集合	封装内部结构与对象引用

八、四种模式快速选择

题目线索	模式	变化被封装在哪里
同一行为有多个算法/支付/折扣实现	策略	具体策略类
客户不应知道创建哪个具体类	工厂方法	具体创建者/工厂方法
需要整体切换一组配套产品	抽象工厂	具体工厂和产品族
某类必须只有唯一实例	单件	实例创建和访问
遍历聚合但不暴露内部结构	迭代器	迭代状态和容器表示

九、往年题例题与答案

例题 1：2022 多种支付方式

题目：用户有微信支付、支付宝支付，未来还会增加更多支付方式。使用哪种设计模式？有什么好处？使用哪些类设计原则？画类图。

问法	参考答案
使用什么模式？	策略模式。把支付行为定义为 PaymentStrategy 接口，微信、支付宝等分别实现。
类图角色	RechargeService/PaymentContext 持有 PaymentStrategy；WeChatPayment、AlipayPayment、BankCardPayment 实现接口；客户端选择并注入策略。
有什么好处？	消除支付类型 switch；支付方式可独立修改测试；支持新增和运行时替换；核心充值流程保持稳定。
体现哪些原则？	OCP、DIP、SRP、面向接口、组合优于继承、封装变化。

```
PaymentStrategy <|.. WeChatPayment
PaymentStrategy <|.. AlipayPayment
PaymentStrategy <|.. BankCardPayment
RechargeService o-- PaymentStrategy

RechargeService.recharge(amount)
-> paymentStrategy.pay(amount)
```

UML 方向

具体支付类以“虚线 + 空心三角形”指向 PaymentStrategy 接口，表示实现；RechargeService 到 PaymentStrategy 是持有/关联，菱形放在上下文一端可表达聚合。

例题 2：2020 DAO 应对数据库变化

题目：数据库可能从 MySQL 变为 SQL Server，使用何种设计模式实现 DAO，并说明原则、画概念类图。

问法	参考答案
模式	若系统有多种 DAO 需要成套切换，使用抽象工厂模式；若只创建一种 DAO，可用工厂方法。
抽象	定义 UserDao、OrderDao 等产品接口，并定义 DaoFactory 创建这些 DAO。
具体实现	MySQLDaoFactory 创建 MySQLUserDao/MySQLOrderDao；SqlServerDaoFactory 创建对应 SQL Server DAO。
业务层	Service 只依赖 DaoFactory 和 DAO 接口，不直接依赖数据库实现。
原则	DIP、OCP、SRP、面向接口、信息隐藏。

例题 3：优惠券变化的诱导式设计

题目不一定要求说出模式名称，而可能逐步问：如何添加数值折扣和比例折扣？怎样减少修改？类之间如何协作？

- 1 识别变化点：优惠计算算法会增加和修改。
- 2 提取 CouponPolicy 接口，例如 discount(total)。
- 3 ValueCoupon 和 PercentageCoupon 分别实现该接口。
- 4 Order/Recharger 持有 CouponPolicy，并把计算请求委托给它。
- 5 创建和选择具体优惠券放在工厂、配置或客户端，不在上下文中写 switch。
- 6 新增新优惠只增加实现类，体现 OCP；上下文依赖接口体现 DIP；每个策略职责单一体现 SRP。

```
interface CouponPolicy {
    double discount(double total);
}

class ValueCoupon implements CouponPolicy { ... }
class PercentageCoupon implements CouponPolicy { ... }

class RechargeService {
    private CouponPolicy coupon;
    double finalAmount(double total) {
        return total - coupon.discount(total);
    }
}
```

十、设计模式答题模板

步骤	答题内容
识别变化	指出题目中未来会增加、替换或动态配置的部分
提取稳定接口	写出统一方法、参数和返回值
定义角色	接口、具体实现、上下文、客户端/工厂
说明协作	谁选择对象，谁持有接口，谁把请求委托给实现
解释扩展	新增类型只增加哪个类，哪些旧类不改
联系原则	OCP、DIP、SRP、组合优先、面向接口、封装变化
说明代价	类数量、配置复杂度、客户端选择责任、通信成本

最后速记

模式	一句话
策略	封装可替换算法，让变化独立于使用者
工厂方法	把某种具体产品的实例化延迟给子类/工厂方法
抽象工厂	通过统一工厂接口创建一组相互配套的产品
单件	保证一个类只有一个实例并提供访问点
迭代器	顺序访问聚合元素而不暴露内部表示